

Report

0.2

Customer

Epineon



Third Review Round

NAPQES v3 — AEAD Scheme Audit

August 8, 2026

Contents

1. Document Revisions	3
2. Introduction	5
3. System Overview	6
4. Methodology	8
5. Findings	9
5.1. Major	10
5.2. Moderate	13
5.3. Minor	24
6. Conclusion	45
About Us	48
Contacts	48

1. Document Revisions

#	Date	Author	Description
0.1	August 4, 2026	Dmitry Khovratovich	Initial v3-round report. Reconciled the fourteen second-round findings together with the five first-round findings still open or acknowledged (eighteen fixed, one partially fixed, none unresolved, none regressed) and filed twenty findings for the v3 revision.
0.2	August 8, 2026	Dmitry Khovratovich	Added five findings from an independent review pass over the same source (two moderate, three minor), bringing the round to twenty-five. Extended the high-severity tag-input finding with a concrete ciphertext-integrity forgery, establishing that the defect is an authentication break and not only a correctness failure, and with the direction its repair must take. Extended the truncation-lemma finding with a counterexample to its converse and with a proof that truncation cannot amplify bias. Three further claims raised in that pass were checked, refuted, and deliberately not filed.

2. Introduction

ALL MODIFICATIONS TO THIS DOCUMENT ARE PROHIBITED. VIOLATORS WILL BE PROSECUTED TO THE FULL EXTENT OF THE U.S. LAW.

Client Comment

IT IS IMPORTANT TO NOTE THAT A SECURITY AUDIT IS NOT A GUARANTEE OF ABSOLUTE SECURITY BUT RATHER A SNAPSHOT OF THE SYSTEM'S SECURITY POSTURE AT A SPECIFIC POINT IN TIME. WHILE WE STRIVE TO UNCOVER ALL POTENTIAL ISSUES, THE EVOLVING NATURE OF BLOCKCHAIN TECHNOLOGY AND DEFI MEANS NEW VULNERABILITIES CAN EMERGE.

3. System Overview

NAPQES (Noise-Augmented Post-Quantum Encryption Scheme) is an Authenticated Encryption with Associated Data (AEAD) scheme whose security is claimed to reduce entirely to the pseudorandomness of HMAC-SHA256, deliberately avoiding the finite-field and permutation structures used by AES-GCM and ChaCha20-Poly1305.

Version 3 keys two independently sampled secrets. The arithmetic-layer key $\mathbf{k} = (k_1, \dots, k_K)$ is an ordered tuple of K distinct primes drawn uniformly without replacement from the set $\mathcal{P}$ of primes in $[10^6, 9.9 \times 10^6]$ (default $K = 10$), serialised as five-byte big-endian encodings; the derivation secret $\text{sk} \in \{0, 1\}^{256}$ is drawn uniformly and independently of it. sk is the only secret that ever keys an HMAC call, and $\mathbf{k}$ is used only, and un-keyed, by the arithmetic token map, so no security theorem in the paper depends on the min-entropy of $\mathbf{k}$.

All internal values are produced by a domain-separated derivation

$$\text{Derive}_d(\text{sk}, N, \text{ctx}) = \text{HMAC}(\text{sk}, d \parallel N \parallel \text{ctx})$$

over nine domains covering the noise schedule, the padding codepoints, the per-position addends, the keystream and the authentication tag. The nonce is synthetic, in the manner of RFC 5297 SIV:

$$N = \text{HMAC}(\text{sk}, \text{exttt0x0A} \parallel \text{be8}(|A|) \parallel A \parallel M)[0:16],$$

so encryption is a pure function of the associated data and the message, and accidental nonce reuse across distinct inputs is a collision event rather than a sampling failure.

A plaintext of n codepoints is padded to a power-of-two bucket B , namely

$$B = \max(16, 2^{\lceil \log_2(n+1) \rceil}),$$

giving $R = B + 2$ real tokens after a two-codepoint length prefix. Each padded codepoint c becomes a token $c \cdot k + a$ for a prime $k \in \mathbf{k}$ and a derived addend $a \in [1, k - 1]$, with pseudorandom noise tokens interleaved among the real ones. Runs of consecutive noise tokens are capped at $\text{MAX_NOISE_RUN} = 19$, and the stream is then padded with filler tokens to exactly $20R$ tokens, so the token count is a deterministic function of the padding bucket alone. Tokens are serialised in fixed-width eight-byte fields, masked with a keystream derived from domain $0x07$, and authenticated by a 32-byte tag computed over the length-prefixed associated data and the masked blob. The ciphertext is $C = N \parallel \widetilde{B} \parallel T$, of length $16 + 160(B + 2) + 32$ bytes.

Version 3 is a rewrite rather than an edit of the previously audited version 2: some sixty-nine per cent of the source is gone. Removed entirely are the legacy version-7 construction together with its wire format and its three security theorems, the online-AE

streaming format, the LEB128 variable-length encoding, the traffic-analysis lemma, the key-guessing term and its minimum- K analysis, and the whole of the embedded audit-response narration — some twenty-five Audit finding remark blocks and roughly ninety inline fix references, of which the current source retains none. What remains is a single construction analysed under a single set of assumptions, with IND-CPA-det, INT-CTXT and IND-CCA-det claims stated against the standard, uniform-key HMAC-SHA256 PRF assumption and with no key-entropy term.

Scope of this round. The subject of this third round is the single self-contained $\text{\LaTeX}$ source `napseq-eprint-v3.tex` (1,259 lines) and the eighteen-page PDF compiled from it. The document contains no `\input` or `\include` directives and no ignored or commented-out normative text, so its entire content appears in the compiled PDF, and every finding below describes a defect present in the document the client distributes. The diff base is the previous round's `napseq-eprint-v2.tex`. No implementation artifacts were submitted for this round, so no specification-to-code cross-check was performed; the implementation-side items deferred by the client's response document, and the unauthenticated-streaming caveat that version 3 drops along with the streaming format it described, remain unverified and are carried into the next round.

Finding locations. The source file could not be registered as an artifact on the audit server for this round, owing to a request-size limit in the upload interface. The findings below therefore cite each location in the body of the finding, by source line interval and by page of the compiled PDF, rather than through the quoted source excerpts this report normally reproduces beneath each finding. Every citation was resolved against the same file the client supplied and is reproducible from it.

Client Comment

4. Methodology

The methodology utilized by ABDK Consulting is a combination of established audit tactics and methods specifically tuned for the requirements and technological stack of the NAPQES v3 — AEAD Scheme Audit project. This approach is flexible and adapted based on the specific project structure, technologies used, and the client's expectations for the security review.

Our audit process incorporates several distinct phases of analysis:

General Code Assessment

The codebase is first reviewed for clarity, consistency, style, and adherence to established best practices. This phase includes checking indentation standards, naming conventions (e.g., following a style guide), appropriate commenting, code duplication, and confusing or irrelevant naming. At this phase, we establish an understanding of the overall system structure and codebase organization.

Entity Usage Analysis

Usages of various entities (contracts, libraries, functions, state variables) defined in the code are analyzed. This includes both internal usages from other parts of the system as well as potential external interactions. We verify that entities are defined with appropriate visibility scopes (internal, external, public, private) and access levels. At this phase, we understand the overall system architecture and how different parts of the code are related and interact with external interfaces.

Access Control Analysis

For those entities that can be accessed externally or manage sensitive operations (like data submission or role management), access control measures are rigorously analyzed. We check that access control mechanisms are correctly implemented, relevant to the intended function, and robust against circumvention. At this phase, we understand the defined user roles, permissions logic, and what digital assets the system ought to protect.

Code Logic Analysis

The specific code logic of critical functions is analyzed for correctness, efficiency, and security. We verify that the code actually performs its intended function as described in the system overview, that algorithms are optimal for on-chain execution, and that proper data types are used to prevent overflow/underflow issues. We also ensure that external libraries used are up to date and relevant. At this phase, we understand the data structures used and their specific purposes.

We classify all identified issues uncovered during these phases by the following severity levels:

Critical

Directly affects the smart contract functionality and may cause a significant loss of assets (e.g., permanent lock of funds, unauthorized minting, total system compromise) or a complete failure of the platform's core invariants.

Major

Is either a solid performance problem or a sign of significant misuse potential: a slight code modification or environment change may lead to loss of funds or data integrity issues. Sometimes it is an abuse of unclear code behavior which should be double-checked by the client for specific edge-case business logic.

Moderate

Is not an immediate functional problem but rather suboptimal performance in edge cases, an obviously bad code practice, or a situation where the code is correct only in certain expected business flows (lacking robust handling of all possible inputs).

Minor

Covers code style improvements, adherence to best practices that do not immediately present a security risk, and general suggestions for code clarity, gas efficiency, or future-proofing the system.

5. Findings

We have completed our technical audit of the NAPQES v3 — AEAD Scheme Audit codebase and provided the client with a detailed list of recommendations and identified vulnerabilities. The following statistics summarize the issues discovered and their current resolution status at the time of this report's finalization.

	Found	Fixed
Major	1	0
Moderate	8	0

Minor

Found
16

Fixed
0

Fixed 0 out of 25 issues reported.

5.1. Major

CVF-1: open

Category: **behavior**

Decryption recomputes the authentication tag over a 4-byte associated-data length prefix while encryption computes it over an 8-byte prefix, so no honest ciphertext verifies, the stated correctness property is false, and the ciphertext-integrity theorem is false as well

Encryption computes the tag as $T = \text{Derive}_{0x03}(\text{sk}, N, \text{be8}(|A|) \| A \| \widetilde{B})$. That 8-byte big-endian length prefix is used consistently in the derivation-schedule table (row 0x03, lines 286-308, PDF p. 5), in the wire-format equation (lines 488-490, PDF p. 8), and in step (6) of the encryption algorithm (line 562, PDF pp. 8-9). The declared associated-data space depends on it: the algorithm triple states that "any $|A| < 2^{64}$ bytes is supported unambiguously, with no practical ceiling", which is true only for an 8-byte field.

Decryption does something else. Step (4) of the decryption algorithm reads $T = \text{Derive}_{0x03}(\text{sk}, N, \text{be4}(|A|) \| A \| \widetilde{B})$ — a **4-byte** prefix (line 574, PDF p. 9). Since the two derivations feed different byte strings into the same HMAC key and domain, they produce unrelated 32-byte outputs for every associated data of every length, including the empty string. Step (5) then compares the recomputed tag against the transmitted one and outputs the rejection symbol on mismatch. Read literally, the specified decryption algorithm rejects **every** ciphertext the specified encryption algorithm produces, and the correctness property asserted a few lines later (lines 583-589, PDF p. 9) is therefore false for the scheme as written. The accompanying correctness argument does not notice, because it reasons only about the token-emission loop and its inversion and takes tag agreement as a hypothesis ("token emission and inversion are exact inverses whenever $T = T'$ ").

The same stale 4-byte form reappears independently inside the ciphertext-integrity proof, which writes the forgery attempt's tag input as $x^* = 0x03 \| N^* \| \text{be4}(|aad^*|) \| aad^* \| \widetilde{B}^*$ (line 847, PDF p. 12). This matters more than a typo, because the proof's Case

2 turns entirely on injectivity of that encoding, and the injectivity it appeals to is the one established by the domain-separation lemma (lines 310-332, PDF p. 5) — which is proved for the 8-byte prefix.

The mismatch is an integrity break, not only a correctness failure. Because the decryptor consumes a 4-byte length field where the encryptor emitted an 8-byte one, the four leading zero bytes of the 8-byte prefix can be re-attributed from the length field into the body, and the boundary between the associated data and the masked blob can be moved. Concretely: query the encryption oracle on (A_j, M_j) with $|A_j| \equiv 156 \pmod{160}$, obtaining $c_j = N_j \| \widetilde{B}_j \| T_j$ where T_j is the tag over $0x03 \| N_j \| \text{be8}(|A_j|) \| A_j \| \widetilde{B}_j$. Then submit the verification query with empty associated data and $c^* = N_j \| \widetilde{B}^* \| T_j$, where $\widetilde{B}^*$ is the low four bytes of $\text{be8}(|A_j|)$ followed by A_j followed by $\widetilde{B}_j$. Decryption builds $x^* = 0x03 \| N_j \| \text{be4}(0) \| \widetilde{B}^*$, and since $\text{be4}(0)$ concatenated with the low four bytes of $\text{be8}(|A_j|)$ is exactly $\text{be8}(|A_j|)$ for any $|A_j| < 2^{32}$, this is byte-identical to the string the encryptor tagged. The recomputed tag equals T_j and decryption accepts. The length congruence is what makes the parse check pass: it forces the token count $(4 + |A_j|)/8 + 20R_j$ to be an integer divisible by 20, and it is the only constraint the attack places on the adversary.

The pair is fresh under the theorem's own freshness definition — the ciphertext differs and the associated data differs — and the attack succeeds with probability approximately 1 using one encryption query and one verification query, against an asserted bound of $\text{Adv}^{\text{PRF}}(\mathcal{B}_2) + q_v/2^{256}$. The theorem "INT-CTXT" is therefore **false for the scheme as literally specified**, not merely unsupported by its proof. The recovered plaintext is whatever the re-split body deserialises to, so this is an authentication break rather than a decryption oracle; that is what an integrity notion forbids, and it is what makes the severity of this finding unambiguous.

The fix must widen, not narrow. Both remaining sites should move from the 4-byte prefix to the 8-byte one, and not the reverse. The 8-byte form is what the encryptor emits, what the synthetic-nonce derivation in domain $0x0A$ consumes (line 328), and — decisively — what the domain-separation lemma actually proves injective (line 324, which reasons that the fixed-width 8-byte length "fixes $|A|$ ", so the split between A and $\widetilde{B}$ is unambiguous). Narrowing every site to 4 bytes would instead break that lemma for associated data of length at least 2^{32} and would require its proof to be reworked, and it would contradict the declared associated-data space of $|A| < 2^{64}$.

We checked the counterfactual explicitly. With lines 574 and 847 both on the 8-byte prefix, the tag input parses by fixed offsets — domain byte, 16-byte nonce, fixed-width 8-byte length, then the associated data and the blob at boundaries the length determines — so the map from tag input to the triple of nonce, associated data and blob is injective, and the three evasions worth trying all fail: changing the associated-data length changes the fixed-width field that is read first; the unused high bytes of the 8-byte prefix become inert once both parties agree on the width; and reallocating bytes between the nonce, the blob and the tag changes the blob and hence the recomputed tag. Step 6 of the integrity proof, which currently reads an equation between a 4-byte-prefixed string and an 8-byte-prefixed string as though it were an equation within one encoding, then becomes sound, and the two steps that inherit from it recover with it. **After those two edits, "The-**

orem 4.10" is true as stated, bound included — not merely no longer broken by this particular attack.

Source anchors (pending artifact upload): lines 568-580, PDF p. 9; lines 845-878, PDF p. 12; lines 583-602, PDF p. 9; lines 310-332, PDF p. 5.

Recommendation

Change both remaining occurrences to the **8-byte** prefix — step (4) of the decryption algorithm (line 574) and the tag input in the ciphertext-integrity proof (line 847) — and not the other way round. The first edit is the security fix, since it is what closes the forgery; the second aligns the proof with the corrected construction, after which the proof's injectivity step is sound and the theorem holds as stated. Narrowing the other sites to 4 bytes is not an acceptable alternative: it would invalidate the domain-separation lemma's injectivity argument for associated data of length at least 2^{32} and contradict the declared associated-data space.

Then state the tag input once, as a named quantity, and refer to that name from the derivation table, the wire-format section, both algorithm descriptions and the integrity proof, so that no future edit can leave the encryptor and the decryptor disagreeing.

Since the correctness property is currently false as written, it is worth restating it as a proposition with a proof that begins by checking that the two tag inputs coincide, rather than assuming tag agreement.

Finally, confirm which prefix width the Python, Rust and C implementations use and publish that as the normative answer, because the three of them cannot all match a document that contains both widths. If any implementation follows the document's decryption step literally, it is vulnerable to the forgery above and should be treated as a fix to ship rather than an editorial correction.

Client Comment

5.2. Moderate

CVF-2: open

Category: **architecture**

The noise-injection and prime-token layer is not required by any proven or claimed property, yet it is the sole cause of the scheme's 160-fold ciphertext expansion, and the one justification offered for it cites a lemma that refutes it

The construction has two layers. A pseudorandom layer keyed by sk produces the noise schedule, the padding codepoints, the per-position addends, the keystream and the tag. An arithmetic layer maps each padded codepoint c to the integer $c \cdot k + a$ using a prime k drawn from the secret tuple $\mathbf{k}$, and interleaves noise tokens among the real ones. The paper is now admirably clear that the second layer carries no confidentiality: the remark on the roles of the two secrets (lines 251-270, PDF p. 5) states that "none of the security theorems of Section 4 depend in any way on $H_\infty(\mathbf{k})$ ", and the caveats section (lines 1125-1150, PDF p. 15) goes further and shows that an adversary holding sk alone recovers any plaintext codepoint by trial division over the roughly 586,000 candidate primes, "an $O(|\mathcal{P}|)$ trial-division search that succeeds in microseconds". Both statements are correct, and we confirmed them independently.

What is left unexamined is the consequence. If no theorem depends on the arithmetic layer, and if the layer provides no confidentiality margin once sk is known, then removing it costs the paper nothing that it currently proves. Deleting the noise schedule, the token map and the fixed-width token serialisation, and masking the padded plaintext directly with the same domain- 0×07 keystream, would leave every one of the three security theorems standing with the same bounds and the same proofs, because each of them reduces solely to the pseudorandomness of the keystream, the tag and the synthetic nonce. It would also collapse the ciphertext from roughly 160 bytes per plaintext codepoint to a little over one, which is the entire practical cost of the design.

The remark on key roles does try to justify keeping the layer, and the justification does not survive inspection. It says the default $K = 10$ "is retained as the default purely for the traffic-analysis-resistance contribution of the multi-prime token construction (Lemma 3.8 and the discussion in Section 11)". Lemma 3.8, on the deterministic length of the ciphertext, proves that $|C|$ is a function of the padded-length bucket **alone** — explicitly "never through sk , N , A , or the specific codepoints of M ", and therefore never through $\mathbf{k}$ or through the number of primes in it. The cited lemma is exactly the statement that the multi-prime construction contributes nothing observable to ciphertext length, so it cannot also be the authority for a traffic-analysis benefit derived from that construction. The previous version did carry a separate traffic-analysis lemma that the remark could have leaned on, and this round removed it — correctly, since its hypothesis was unsatisfiable

for a message-derived nonce — but the citation that depended on it was left pointing at the replacement.

This is the surviving residue of the first-round finding that all confidentiality rests on the keystream and that the prime-token layer contributes nothing. That finding was recorded as fixed on the strength of the honest new caveat, and the caveat is indeed honest. The residue is the unsupported claim that remains attached to the parameter K , and the larger question the paper still does not ask.

Source anchors (pending artifact upload): lines 251-270, PDF p. 5; lines 410-449, PDF pp. 6-7; lines 504-525, PDF p. 8; lines 1125-1150, PDF p. 15.

Recommendation

Either delete the traffic-analysis justification for K from the remark on key roles, leaving the parameter as an explicitly unmotivated default, or state and prove whatever property the multi-prime layer is actually intended to provide, against a defined adversary.

If no such property can be stated, we recommend saying so plainly and adding a short subsection that compares the specified construction with the same construction minus the arithmetic layer, showing which theorems change (none, by our reading) and what the expansion becomes. A reader deciding whether to deploy this scheme needs that comparison, and a design whose dominant cost buys a property nobody can state is unlikely to survive external review without it.

Client Comment

CVF-3: open

Category: **algorithm**

Ciphertext expansion is stated as a fixed 20-fold in four places, but the on-wire expansion is 160-fold asymptotically and between 160 and 320-fold across padding buckets, as the paper's own minimum-ciphertext figure shows

Every message is padded to $R = B + 2$ real tokens, where B is the power-of-two padding bucket, and the token stream is then padded to exactly $R \cdot (\text{MAX_NOISE_RUN} + 1) = 20R$ tokens. This is the repair for the previous round's inconsistent 4-to-20-fold range, and as a determinism claim it is correct and we verified it. The multiplier, however, counts **tokens per codepoint**, and each token is serialised into a fixed-width 8-byte field. The wire cost is therefore $8 \times 20 = 160$ bytes per padded codepoint, not 20.

The paper states the figure as a ciphertext-expansion factor four times, in each case against schemes whose expansion is quoted as 1x, which is unambiguously a byte-for-byte comparison: the comparison table's expansion row reads "20× (fixed)" against "1×" for AES-GCM and ChaCha20-Poly1305 (line 992, PDF p. 14); the surrounding discussion calls it "a hard, deterministic 20× per real codepoint" (lines 1000-1009, PDF p. 13); the caveats section repeats "fixed at exactly 20× per real codepoint" (lines 1110-1112, PDF p. 15); and the conclusion refers to "the fixed 20× ciphertext expansion" (lines 1174-1177, PDF p. 16).

The paper's own arithmetic refutes it. The wire-format section computes the minimum well-formed ciphertext by taking the shortest possible message, which pads to $B = 16$ and $R = 18$, giving $N_{\text{tok}} = 360$, $|B| = 2880$ bytes and $|C| = 2928$ bytes (lines 496-502, PDF p. 8). We reproduced that number. A ciphertext of 2,928 bytes for a single-codepoint plaintext is an expansion of 2,928-fold, and even at the most favourable point of a bucket it is 160-fold. Concretely, using $|C| = 16 + 160(B(n) + 2) + 32$: a 1-codepoint message gives 2,928 bytes; 16 codepoints give 5,488 bytes, or 343-fold; 1,000 codepoints give 164,208 bytes, or 164-fold; the maximum 65,535-codepoint message gives 10,486,128 bytes, or 160-fold. Because B rounds up to a power of two, the factor oscillates between roughly 160 at the top of a bucket and roughly 320 just past a bucket boundary, and it is unboundedly larger for messages shorter than 16 codepoints.

The gap between 20 and 160 is not a rounding difference; it is the difference between an overhead a deployment might absorb and one that rules the scheme out for most transports. The figure also feeds the trade-off argument that immediately follows it, which concludes that "this overhead is acceptable in applications where the traffic pattern does not rely on ciphertext length confidentiality beyond the padding bucket" — a conclusion drawn about an overhead an order of magnitude smaller than the real one.

Source anchors (pending artifact upload): line 992, PDF p. 14; lines 1000-1009, PDF p. 13; lines 1110-1112, PDF p. 15; lines 1174-1177, PDF p. 16; lines 496-502, PDF p. 8.

Recommendation

State expansion in on-wire bytes per plaintext codepoint and separate it from the token multiplier. We suggest saying that the token count is exactly $20R$ tokens for $R = B(|M|) + 2$, that each token occupies 8 bytes, and that consequently $|C| = 48 + 160(B(|M|) + 2)$ bytes, which is 160 bytes per codepoint at the top of a padding bucket, up to about 320 bytes per codepoint just after a bucket boundary, and 2,928 bytes for any message of at most 15 codepoints.

Correct all four sites, including the comparison table row, whose neighbouring 1x entries make the units unambiguous. Then revisit the trade-off paragraph and the conclusion with the corrected magnitude, since the recommendation a reader should draw from 160-to-320-fold expansion is not the one currently drawn from 20-fold.

Client Comment

CVF-4: openCategory: **architecture**

The confidentiality notion actually proved is a single-challenge, no-repeat game that is strictly weaker than the deterministic-authenticated-encryption notion the paper repeatedly invokes by name

This round replaced the unrestricted left-or-right game, which a deterministic scheme cannot satisfy, with a restricted notion (lines 683-708, PDF p. 10). The replacement is the right move and the resulting theorem is sound; we checked every step. What has not been adjusted is the surrounding language, which continues to place the scheme in the company of a substantially stronger notion.

The notion as defined gives the adversary an encryption oracle, then **one** challenge query on a pair (m_0, m_1) of equal codepoint length, then further oracle access, and asks it to guess the challenge bit. Two restrictions apply: no repeated (A, M) encryption query in either phase, and no encryption query on either challenge message. The deterministic and misuse-resistant notion of Rogaway and Shrimpton, cited three times in support (lines 180-184 PDF p. 3, lines 678-681 PDF p. 10, and the closing sentence at lines 700-704 PDF p. 10), is not this game. It asks for indistinguishability of the encryption oracle from a random-bits oracle under adaptive, unlimited queries — a notion that implies the game above for every choice of challenge pair simultaneously, and that additionally rules out any correlation an adversary might detect across many ciphertexts rather than one. AES-GCM-SIV, invoked as the model for the design (lines 463-468 PDF p. 7 and lines 1011-1019 PDF p. 13), is analysed under that stronger notion.

The restriction against repeated queries is also presented as inherited: the definition describes both restrictions as "standard for deterministic/misuse-resistant AE" with a citation to Rogaway and Shrimpton. The second restriction is standard and necessary. The first is not part of that notion at all — in an indistinguishability-from-random-bits game a repeat is permitted and merely useless, since the oracle is a function on both sides. Forbidding repeats here is harmless, but it is an additional restriction on the adversary, so presenting it as inherited overstates how much of the standard notion is being met.

We want to be precise about what is and is not being claimed. We are not claiming an attack: we found no adversary that beats the defined game, and we found none that beats the stronger notion either. The finding is that the paper's headline claim of misuse resistance, and its comparison of itself to AES-GCM-SIV on that axis, are supported by a theorem about a weaker game than the one those comparisons refer to. Given that the construction is a synthetic-nonce design whose whole selling point is misuse resistance, the gap between the notion invoked and the notion proved is the kind of thing a reader is most likely to rely on and least likely to check.

Source anchors (pending artifact upload): lines 683-708, PDF p. 10; lines 672-681, PDF p. 10; lines 63-78, PDF p. 1; lines 180-184, PDF p. 3; lines 1011-1019, PDF p. 13.

Recommendation

Either prove the stronger statement or narrow the prose.

The stronger statement looks reachable with the machinery already present: the hiding argument already establishes that a fresh nonce makes an entire ciphertext uniform and independent of the message, so extending it to indistinguishability from random bits under adaptive queries is a matter of a union bound over queries rather

than over a single challenge, at the cost of a $q^2/2^{128}$ term the bound already carries. If that is done, the restriction against repeated queries can be dropped and the comparison with AES-GCM-SIV becomes accurate.

If it is not done, then say explicitly that the proved notion is a single-challenge restricted game, note that full deterministic-authenticated-encryption security in the sense of Rogaway and Shrimpton is not established, and stop describing the restriction against repeats as standard for that notion.

Client Comment

CVF-5: open

Category: **architecture**

IND-CCA-det is claimed in the abstract, the contributions and the conclusion but is never defined, and the theorem asserting it is a proof sketch that attributes to Bellare and Namprempe a composition result their paper does not state

Chosen-ciphertext security is one of the paper's three headline results. It appears in the abstract (lines 68-71, PDF p. 1), in the third contribution (lines 123-126, PDF p. 3), as a named theorem with a concrete bound (lines 883-898, PDF p. 12), and in the conclusion (lines 1165-1168, PDF p. 16). There is no definition of it anywhere in the document. The only description of the game appears inside the proof, as a parenthetical: "define H_0 as the real IND-CCA-det game (encryption and adaptive decryption oracles, with only the exact pair (c^*, A^*) forbidden from the decryption oracle)". The theorem statement adds that the adversary respects the confidentiality notion's two restrictions "on its own encryption-oracle queries", which leaves open whether they bind after the challenge is issued, and the decryption oracle's freshness condition is never stated as part of a notion at all. Without a definition, the theorem's meaning is fixed only by that parenthetical, and a reader cannot check whether the bound is the right bound for the game they care about.

The proof is also labelled "Proof sketch" — the only one in the paper — for the result that a reader is most likely to depend on in practice.

The sketch opens by invoking a composition theorem: "We apply the Bellare-Namprempe composition theorem: a scheme that is both IND-CPA-det and INT-CTXT is IND-CCA-det." Bellare and Namprempe's 2000 paper proves that indistinguishability under chosen plaintext together with ciphertext integrity implies indistinguishability under chosen ciphertext, for the **standard, probabilistic** notions. It does not treat deterministic encryption, does not define restricted-query variants, and therefore does not state the sentence attributed to it. The restricted-query analogue is widely believed

and is the natural adaptation, but the citation as written asserts that the composition step is already in the literature when the specific form used here is not.

The concrete argument that follows the citation is, to its credit, the right argument, and its two steps hold: the real game and the table-backed game agree except on a fresh valid forgery, whose probability the integrity theorem bounds, and the table-backed game reveals nothing the adversary could not reconstruct, so it reduces to the confidentiality game. We also confirmed the detail that is easy to get wrong — that the forger constructed for the first step must itself issue the encryption query producing the challenge ciphertext, which the sketch does say. So we expect the bound is correct. The finding is that a headline claim currently rests on an undefined notion, a sketch, and a citation that does not carry the statement attached to it.

Source anchors (pending artifact upload): lines 883-924, PDF p. 12; lines 68-71, PDF p. 1; lines 123-126, PDF p. 3; lines 1165-1168, PDF p. 16.

Recommendation

Add a definition of the chosen-ciphertext notion, in the same style as the confidentiality definition, stating explicitly the encryption oracle, the decryption oracle, which pairs the decryption oracle refuses, whether the restriction against repeated encryption queries binds in both phases, and the advantage.

Then promote the sketch to a full proof against that definition; the argument is already three quarters written and the missing quarter is bookkeeping.

Replace the composition citation with either a statement of the restricted-notion composition result proved in place, which is short, or a citation that actually covers deterministic authenticated encryption, with a sentence explaining why the restricted notions compose the same way. If the sketch is kept as a sketch, say which steps are omitted.

Client Comment

CVF-6: open

Category: **typing**

The message space is defined as Unicode codepoints while the nonce is computed over UTF-8 bytes and the reference interface takes bytes, and no mapping between the three is specified

Three incompatible notions of "message" appear within a few pages, and the specification never reconciles them.

The algorithm triple defines the message space as "finite sequences of Unicode codepoints of length at most

$$\text{MAX_PLAINTEXT_CODEPOINTS} = 2^{16} - 1$$

" (lines 541-544, PDF pp. 8-9), and the padding and token layers operate on codepoints throughout: the length prefix is two codepoints, the bucket B counts codepoints, padding codepoints are drawn in $[32, 126]$, and each codepoint c becomes a token $c \cdot k + a$ (lines 357-373 and 375-449, PDF pp. 6-7). The synthetic-nonce definition, by contrast, takes M in the message space "(encoded as UTF-8 bytes)" and hashes those bytes (lines 454-461, PDF p. 7). The implementation section then names the entry points as `encrypt_bytes_v8` and `decrypt_bytes_v8` in all three languages (lines 1026-1049, PDF p. 14), and the known-answer-test corpus is said to cover "binary AAD" — so the interface a caller sees is byte-oriented.

No mapping is given in either direction. A caller with an arbitrary byte string has no specified way to obtain a codepoint sequence, and most byte strings are not valid UTF-8, so they are simply outside the defined message space even though the interface accepts them. In the other direction, the declared message space admits codepoints in U+D800 through U+DFFF, which have no UTF-8 encoding at all; for a message containing one, the nonce derivation is undefined, and so is the whole encryption. The specification also never bounds the codepoint values that the token map must accommodate, so the fixed-width 8-byte token field is never justified. It happens to be sufficient — the largest possible token is U+10FFFF times 9,899,999 plus an addend, about 1.10×10^{13} , comfortably below 2^{64} — but the paper's notation section only says of its fixed-width encodings that " x is assumed to fit", and that obligation is never discharged.

The practical consequence is that the document does not determine what any of the three reference implementations does with a byte string that is not valid UTF-8, nor whether they agree with each other. That is precisely the question a cross-language known-answer-test corpus is supposed to settle, and it is left open. It is also the residue of the first-round finding that the codepoint unit was never defined: this round defines the unit but not its relationship to the bytes that callers and the nonce derivation actually consume.

Source anchors (pending artifact upload): lines 541-544, PDF pp. 8-9; lines 454-461, PDF p. 7; lines 357-373, PDF p. 6; lines 1026-1049, PDF p. 14; lines 198-200, PDF p. 4.

Recommendation

Pick one domain and make the rest derived. The cleanest choice for an authenticated-encryption scheme is to define the message space as arbitrary byte strings, specify how bytes are mapped into the codepoint sequence the token layer consumes — for instance by treating each plaintext byte as a codepoint in $[0, 255]$, which also removes the surrogate problem and lets the length cap be stated in bytes — and then state the nonce derivation over the same byte string, so that all three layers agree by construction.

If the codepoint domain is retained instead, then state the required encoding explicitly, exclude surrogates from the message space, define what the byte-oriented entry points do when handed input that does not decode, and say which error they raise.

In either case, add the one-line calculation showing that every token fits in 8 bytes, and state the length cap once, in the unit that the interface actually enforces.

Client Comment**CVF-7:** openCategory: **documentation****The comparison table asserts that NAPQES is FIPS-approved, which conflates the approval status of a primitive with that of a scheme built from it**

The comparison table's second row is labelled "FIPS-approved" and gives NAPQES a bolded "Yes", AES-GCM "Yes", ChaCha20-Poly1305 a bolded "No", and Ascon "No (pending)" (line 988, PDF p. 14). The row's units are schemes: every other entry in that column is an authenticated-encryption scheme, and AES-GCM's "Yes" is true because AES-GCM itself is specified as an approved mode in NIST SP 800-38D. NAPQES is not specified in any FIPS publication or NIST Special Publication, has no CAVP algorithm listing, and has not been submitted to any standardisation process. Its "Yes" is therefore not true in the sense the row establishes.

The claim the paper can defend is the one it makes elsewhere and makes correctly: the introduction says the only primitive in the construction is HMAC-SHA256, "approved under NIST FIPS 198-1 and FIPS 180-4" (lines 101-107, PDF p. 3), and the conclusion says the scheme "relies solely on FIPS-approved primitives" (lines 1158-1161, PDF p. 16). Both are accurate. Building a new mode out of an approved primitive does not make the mode approved, and in a FIPS 140-3 validation the distinction is the whole question — an unapproved mode over an approved primitive lands outside the approved-mode boundary. That this matters concretely here is clear from the client materials accompanying this round, which include a module-boundary document, a security policy, a CAVP-evidence document and a key-management document; a reader who has those in hand and then reads a bolded "Yes" in this row will draw the wrong conclusion about where NAPQES sits.

The row is also the paper's most prominent comparative claim, since it is one of only two rows where NAPQES is bolded against a competitor's contrasting entry, and the competitor it beats there is ChaCha20-Poly1305, a widely deployed and thoroughly analysed scheme. The comparison as drawn invites the reading that NAPQES has cleared a bar that ChaCha20-Poly1305 has not, when in fact neither has, and one of them is standardised in RFC 8439 while the other is a preprint.

Source anchors (pending artifact upload): line 988, PDF p. 14; lines 101-107, PDF p. 3; lines 1158-1161, PDF p. 16.

Recommendation

Relabel the row to what is actually being compared. "FIPS-approved primitives only" with a "Yes" for NAPQES and AES-GCM and a "No" for ChaCha20-Poly1305 would be accurate and would preserve the point the authors want to make.

If the row is kept as an approval-status row, the NAPQES entry must become "No", with a footnote that its primitive is approved but the mode is not, and the same footnote should say that no validation has been sought.

We would also drop the bolding from this row, since it presents a definitional advantage as an evaluated one, and add a sentence in the same section stating plainly that NAPQES is an unapproved mode and would fall outside an approved-mode boundary in a FIPS 140-3 validation.

Client Comment

CVF-21: open

Category: **algorithm**

The comparison section denies the birthday-bound nonce-collision risk that the paper's own confidentiality theorem quantifies, and the resulting limit of roughly 2^{64} messages per key is never disclosed

The theorem "IND-CPA-det" bounds the adversary's advantage by $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B}_1) + q^2/2^{128}$ (lines 732-745, PDF p. 10), where the second term arises from collisions of the 128-bit synthetic nonce across distinct associated-data and message pairs. That term is not a proof artefact but a genuine attack event: two colliding pairs share the entire keystream and every nonce-keyed derivation, so the exclusive-or of the two masked token blobs equals the exclusive-or of the two unmasked blobs, the noise and filler tokens cancel, and the positions of the real tokens together with the exclusive-ors of their values are exposed — substantial plaintext leakage from a classical two-time pad.

The bound therefore becomes vacuous at roughly $q \approx 2^{63.5}$ encryptions under one key, which is the standard birthday-type data limit for a 128-bit synthetic nonce and is the same limit AES-GCM-SIV carries. The paper nowhere translates this into practical guidance, and the comparison section actively contradicts it, stating that accidental reuse across distinct pairs is "a negligible HMAC-SHA256-collision event ... rather than a birthday-bound or DRBG-failure event" (lines 1011-1019, PDF pp. 13-14). That framing is incorrect: the event is exactly a birthday collision on a 128-bit value, and the confidentiality theorem prices it as such three pages earlier. A reader who takes the comparison section at face value would conclude that no per-key message limit applies, when one does.

We note for completeness that the proof of "Theorem 4.7" is itself sound at every load-bearing step — the game hops, the collision accounting and the one-time-pad step were each re-verified — so this finding is about what the paper omits and contradicts around the bound, not about the bound's derivation.

Source anchors (pending artifact upload): lines 1011-1019, PDF pp. 13-14; lines 732-745, PDF p. 10.

Recommendation

Remove or reword the "rather than a birthday-bound event" clause so it no longer contradicts the theorem, and state explicitly that confidentiality degrades by $q^2/2^{128}$, giving approximately 64 bits of data-complexity security.

Add a concrete per-key message cap with the margin it buys — for example $q \leq 2^{48}$ for a collision margin of 2^{-32} — and place it alongside the other operational limits in the caveats section rather than leaving it implicit in a bound.

Client Comment

CVF-22: open

Category: **architecture**

The ciphertext-integrity theorem bounds an advantage that the paper never defines, and its proof invokes the domain-separation lemma outside the scope that lemma is stated for

The theorem "INT-CTXT" states an upper bound on $\text{Adv}^{\text{INT-CTXT}}(\mathcal{A})$ (lines 829-843, PDF p. 12), but no definition of the ciphertext-integrity experiment appears anywhere in the document. The confidentiality notion is given a full six-step game with its oracles, its two restrictions and its advantage; the integrity notion has nothing comparable, and the term appears only in the abstract, the contributions, the section heading, the theorem, the chosen-ciphertext proof and the conclusion — always asserted, never defined.

As written the reader cannot tell which oracles the adversary is granted, whether the decryption or verification oracle returns the recovered plaintext or only an accept-or-reject bit, whether the q_v verification attempts continue after the first accepting one, or what event precisely constitutes a win. The single inline sentence "Freshness is measured over the pair (c, aad) against the encryption oracle's own outputs" fixes only the freshness relation and not the game. Because the quantity being bounded is undefined, the inequality is not a falsifiable statement and the proof cannot be validated against a fixed target.

A second, smaller defect sits in the same proof and is repaired by the same kind of edit. Case 1 of the proof — the case where the forgery's nonce N^* was never returned by the encryption oracle — argues that the forgery's tag input x^* was never submitted to the underlying function, and cites "Lemma 3.4, Domain separation" for it. That lemma is stated over "every pair of distinct queries ... made anywhere in this construction" (lines 310-313, PDF p. 5), which is a statement about the derivations the construction itself performs; x^* is chosen by the adversary and is not such a query, so the citation reaches beyond what the lemma asserts. The fact the proof needs is nonetheless true and directly provable from positional structure alone: x^* carries N^* at a fixed offset with N^* outside the set of oracle nonces, so it differs from every query-phase domain $0x03$ input at that offset, and it differs from every other domain's input at the leading domain byte. This is therefore a citation-precision defect rather than a hole in the argument, and it survives the unrelated length-prefix repair recorded in CVF-1.

This finding is adjacent to, but distinct from, CVF-5, which records that the chosen-ciphertext notion is likewise never defined. We considered merging the two and did not, because neither recommendation resolves the other's location: adding a chosen-ciphertext definition leaves the integrity game undefined, and vice versa, and the domain-separation scoping defect belongs to neither. If the client prefers a single editorial pass, the natural shape is one new subsection defining all three notions together, which would close CVF-5 and the first half of this finding at once.

Source anchors (pending artifact upload): lines 829-843, PDF p. 12; lines 845-878, PDF p. 12; lines 310-332, PDF p. 5.

Recommendation

Add a formal ciphertext-integrity experiment immediately before the theorem, in the same style as the confidentiality definition: specify the encryption and verification oracles and the verification oracle's exact output behaviour, define the freshness set explicitly as the pairs returned by the encryption oracle, state the win condition as some verification query on a fresh pair returning a value other than the rejection symbol, fix whether verification continues after a success, and define the advantage as the probability of that win. Parameterise the advantage by the scheme and by (q, q_v) so the theorem statement is self-contained.

Separately, restate "Lemma 3.4" over arbitrary inputs rather than over queries the construction makes. Its existing proof uses only the positional structure of the encoded inputs, so it generalises unchanged, and the integrity proof's Case 1 then cites it within its stated scope. The alternative — leaving the lemma as is and giving Case 1 its own two-line positional argument — is equally acceptable.

Client Comment

5.3. Minor

CVF-8: open

Category: **behavior**

Decryption validates only that the ciphertext length is divisible by the token multiplier, with no bucket check, no upper bound, and no check of the recovered length prefix, and it omits the state the token inversion actually needs

Step (3) of decryption computes $N_{\text{tok}} = (|C| - 48)/8$ and rejects unless this is a non-negative integer divisible by $\text{MAX_NOISE_RUN} + 1$, then sets $R = N_{\text{tok}}/20$ (lines 568-580, PDF p. 9). Nothing more is checked. Three checks that the construction implies are missing.

First, no bucket check. An honest ciphertext always has $R = B + 2$ for a padding bucket B in $\{16, 32, \dots, 65536\}$, so R is confined to the thirteen values $\{18, 34, 66, \dots, 65538\}$. The decryptor accepts any R divisible by nothing in particular — $R = 1$, $R = 7$, $R = 100$ — none of which any encryptor can produce. Second, no upper bound: R is unbounded above, so a ciphertext claiming a token count far beyond the 65,538 maximum is accepted as well-formed and drives allocation and derivation work proportional to whatever length was claimed, before any of it can be found inconsistent. Third, no check on the recovered length prefix: step (6) says to “strip the length prefix and padding” but never says that the two-codepoint prefix n must satisfy $n \leq R - 2$, so a ciphertext whose plaintext decodes to an out-of-range n has undefined handling.

None of these is reachable without the key, since the tag is verified at step (5) before any of this parsing has effect, so the security consequence is limited and the severity is low. They matter for two other reasons. They are exactly the class of check that three independent implementations get differently, and the paper offers a cross-language known-answer-test corpus as evidence of agreement, including a negative vector for “under-sized ciphertext” — so the corpus tests the one length check the paper does specify and none of the ones it does not. And the work-amplification point survives tag verification in the sense that an implementation must decide how much to allocate from the claimed length before it can verify anything, which is a decision the specification currently leaves to each implementer.

Two further gaps sit in the same steps. Step (6) says the inversion uses “the same (sk, N) -derived noise-position oracle and addends” but never mentions that it also needs the prime $k[\text{real_idx} \bmod K]$ and a replicated noise-run counter, both of which the emission loop uses and without which inversion is impossible; a reader implementing from the specification alone cannot reconstruct step (6). And the paper’s fixed-width

encodings are introduced with the caveat that " x is assumed to fit" (lines 198-200, PDF p. 4), an obligation never discharged for tokens; it does hold, the largest token being about 1.10×10^{13} , but it should be stated.

Source anchors (pending artifact upload): lines 568-580, PDF p. 9; lines 496-502, PDF p. 8; lines 198-200, PDF p. 4.

Recommendation

Add the missing validation to step (3) and step (6): require R to equal $B + 2$ for some $B = \max(16, 2^j)$ with $2^j \leq 65536$, which is a single table lookup against thirteen values and subsumes both the divisibility and the upper-bound check; and require the recovered length prefix to satisfy $n \leq R - 2$, with rejection otherwise.

State that these checks are performed after tag verification, so that the ordering is normative and no implementation is tempted to reject before authenticating.

Extend step (6) to name the prime selection and the noise-run counter that inversion needs, and add the one-line bound showing every token fits in 8 bytes.

Finally, add negative known-answer vectors for a non-bucket R , an over-large R , and an inconsistent length prefix, so that the corpus covers the checks rather than only the one it already covers.

Client Comment

CVF-9: open

Category: **documentation**

Decryption never re-derives the synthetic nonce, so a valid ciphertext need not carry the nonce the encryptor would have produced, and the correctness argument justifies its key step circularly

The nonce is defined as a deterministic function of the key and the input,

$$N = \text{HMAC}(\text{sk}, 0x0A\|\text{be8}(|A|)\|A\|M)[0:16]$$

, and the paper's misuse-resistance argument rests on that determinism: encryption is "a pure function of (A, M) ", so re-encrypting an identical pair reproduces an identical ciphertext and discloses only message equality. Decryption, however, reads the nonce from the first sixteen bytes of the ciphertext and never checks it against that function (lines 568-580, PDF p. 9). The tag covers the nonce as a byte string, so integrity is unaffected and there is no attack here. What is affected is uniqueness: a party holding sk can produce, for one (A, M) pair, many distinct ciphertexts that all decrypt successfully

to M , simply by choosing a different sixteen-byte nonce and computing the tag over it. The scheme is therefore not tidy, in the sense that valid ciphertexts are not in bijection with (A, M) pairs, and the reader is nowhere told this. Deterministic-authenticated-encryption designs generally either re-derive the synthetic value at decryption or make it the tag itself; this one does neither, and the difference from the RFC 5297 design it invokes goes unremarked.

The correctness argument that accompanies the definition (lines 583-602, PDF p. 9) has three separate presentational defects. It speaks of "the (deterministically re-derivable, once M is recovered token-by-token) N " — but the nonce is not re-derived at any point, and re-deriving it would require M , which requires the nonce, so the justification offered is circular. The true justification is far simpler and is never given: the nonce is transmitted in the clear as part of the ciphertext. Next, the argument closes by appealing to evidence rather than inference: "Correctness is additionally checked empirically by the cross-language KAT corpus and the full regression suite". Testing is valuable and worth reporting, but not inside a proof. Finally, the structure is inverted: a definition environment stating the correctness requirement is immediately followed by a proof environment, so the document proves a proposition it never states.

Note that as the specification currently stands the correctness claim is in any case false, because the decryptor's tag input disagrees with the encryptor's; that is the subject of CVF-1, and this finding concerns what remains once CVF-1 is repaired.

Source anchors (pending artifact upload): lines 583-602, PDF p. 9; lines 568-580, PDF p. 9; lines 454-468, PDF p. 7.

Recommendation

State correctness as a proposition and prove that proposition, beginning by observing that the two tag inputs coincide and that the nonce is transmitted rather than re-derived, so the loop-inversion argument needs no circular appeal.

Move the empirical cross-language evidence out of the proof into the implementation section, where it belongs and where it reads as a strength.

Separately, decide and state whether decryption should re-derive the nonce and reject a mismatch: doing so gives ciphertext uniqueness and makes the comparison to synthetic-IV designs exact, at the cost of one extra HMAC call over the recovered plaintext; not doing so is defensible, but should be stated as a deliberate choice, with a sentence noting that valid ciphertexts are not unique per input pair.

Client Comment**CVF-10:** openCategory: **algorithm**

The truncation lemma states an equality of PRF advantages where only one inequality is proved, comparing an adversary against the truncated function with itself against the untruncated one, and the truncation parameter is overloaded

The lemma asserts that for any fixed truncation length $\tau \leq 256$, writing $\text{trunc}_\tau(x) = x[0:\tau]$, the PRF advantage of an adversary against the truncated function equals its advantage against the full function (lines 651-655, PDF p. 10). The proof gives the correct argument in one direction: an adversary against the truncated composition yields an adversary against the full function, by truncating each oracle answer before continuing unchanged, with the same advantage. That establishes an inequality, and it establishes it between two *different* adversaries — the original one and the derived one. The equality as written puts the same symbol on both sides, and the two sides are not comparable, since an adversary built to consume τ -byte answers cannot be run against a function returning 32-byte answers without the very modification the proof introduces.

The statement at line 654 is in fact ill-typed rather than merely imprecise: on the left the adversary's oracle returns 8τ -bit strings and on the right it returns 256-bit strings, and one machine cannot be both without an implicit input-length convention the paper never states. Under any such convention the equality is false, and the counterexample is concrete. Let G be a secure pseudorandom function into $\{0, 1\}^{8\tau}$ and define $F'(\kappa, x) = G(\kappa, x) \parallel 0^{256-8\tau}$. Then $\text{trunc}_\tau \circ F' = G$ resists every distinguisher, while the one-query distinguisher that outputs 1 exactly when its oracle's answer ends in $0^{256-8\tau}$ achieves advantage at least $1 - 2^{-(256-8\tau)} \approx 1$ against F' itself. The map from an adversary to its derived counterpart is not surjective — adversaries that inspect the dropped bytes are not in its image — which is precisely why only one direction holds.

The parameter is also overloaded in two ways at once. The bound " $\tau \leq 256$ " reads as a bit length, matching the 256-bit output of the primitive, while the definition $\text{trunc}_\tau(x) = x[0:\tau]$ indexes bytes, matching the paper's own $[0:n]$ notation for byte prefixes of a 32-byte digest; taken together they permit truncating a 32-byte digest to 256 bytes. And τ is already in use, thirty lines earlier, for the raw 64-bit integer drawn from the noise-probability derivation (line 344, PDF p. 6), which is a different kind of object entirely.

The defect is the misstatement, not a bias problem. A natural worry about this lemma is that truncation might change how biased a derived value is — that if the full output carries some skew, keeping only a prefix could concentrate it. It cannot. Truncation is

a balanced map: every 8τ -bit image has exactly $2^{256-8\tau}$ preimages, so a uniform full output truncates to an exactly uniform prefix. From that, two things follow, both of which we verified from first principles. Total variation distance from uniform is non-increasing under truncation, by the data-processing inequality together with balancedness. Maximum per-value relative deviation is likewise non-increasing, because each truncated value's probability is a sum of exactly $2^{256-8\tau}$ full-output probabilities, each confined to the same relative band, so the sum stays in that band. Balancedness is essential to both: for a general deterministic post-processing the second claim is false, and a map such as the logical conjunction of two uniform bits manufactures relative deviation out of nothing. Truncation specifically does not.

What remains true is that the scheme's derived values are biased — but by the subsequent reduction modulo a small modulus, not by the truncation. Truncating to 32 bits before reducing does make the final bias about $m/2^{32}$ rather than about $m/2^{256}$, which is a real and large quantitative difference, and the paper's characterisation of that residual bias is mistaken in its own separate way; that is the subject of CVF-23 rather than of this finding.

Nothing downstream breaks. The lemma is used to license three truncations that the construction actually performs — the sixteen-byte nonce, the eight-byte noise threshold and the four-byte addends and characters (lines 665-667, PDF p. 10) — and for each of those, as for the game hops in the confidentiality and integrity proofs, the direction the proof establishes is the direction needed. No argument in the security section relies on the false converse. This is a statement-hygiene finding rather than a soundness one, and it is worth fixing because a lemma that claims more than it proves is exactly what a careful reader stops to check, and because the previous round already asked for this lemma to be added, so its final form should be right.

Source anchors (pending artifact upload): lines 651-667, PDF p. 10; lines 344-348, PDF p. 6.

Recommendation

Restate the lemma over a derived adversary, where it is in fact an exact equality and not merely an inequality: for every adversary $\mathcal{B}$ against the truncated function there is an adversary $\mathcal{B}'$ against the full function, making the same number of queries and running in essentially the same time, with $\text{Adv}_{\text{trunc}_\tau \circ F}^{\text{PRF}}(\mathcal{B}) = \text{Adv}_F^{\text{PRF}}(\mathcal{B}')$; hence $\text{trunc}_\tau \circ F$ is a secure pseudorandom function whenever F is. That is the strongest true statement in the vicinity, it is what the existing proof body already establishes verbatim, and it is what every downstream use consumes.

Fix the units of the truncation parameter, stating it in bytes with the bound $\tau \leq 32$ to match the digest length and the paper's byte-prefix notation, and rename it so it no longer collides with the noise-threshold draw.

It is also worth adding the one-line reason the truncated values inherit uniformity — that truncation is balanced, so it maps a uniform random function to a uniform random function on the shorter output — since that is the fact the game hops actually use and it is currently left implicit.

Client Comment**CVF-11:** openCategory: **algorithm**

The padding-bucket leak is quantified backwards: the ciphertext length reveals which of thirteen buckets a message falls in, at most 3.7 bits, not the stated number of bits of the length

The caveat on padding-bucket leakage states that "ciphertext length reveals the power-of-two padding bucket $\{16, 32, \dots, 65536\}$, leaking up to $\lceil \log_2 n \rceil$ bits of length information" (lines 1114-1123, PDF p. 15). The two halves of that sentence describe opposite quantities. The bucket set it names has exactly thirteen elements, since the message length is capped at 65,535 codepoints, so the ciphertext length is one of thirteen values and an observer learns at most $\log_2 13 \approx 3.7$ bits about the message, whatever n is. The quantity $\lceil \log_2 n \rceil$ is not what is leaked; it is the bucket index, that is, the *identity* of the value learned rather than its information content, and it is also, up to one, the number of bits about n that remain **hidden**: knowing that a message falls in bucket 2^j places n in the interval $[2^{j-1}, 2^j - 1]$, leaving roughly $j - 1$ bits of the length undetermined.

The rest of the caveat is correct and is in fact a genuine improvement on the previous round: the length no longer varies with the secret, the nonce or the associated data, so the averaging attack that the previous round demonstrated — collecting many ciphertexts of one message under varying associated data and cancelling the noise — is dead, and the caveat says so. That makes the misquantification more prominent rather than less, because a reader assessing the residual channel has only this sentence to go on, and the sentence overstates the leak by roughly a factor of four for a typical short message and understates the protection the fix bought.

Source anchors (pending artifact upload): lines 1114-1123, PDF p. 15.

Recommendation

Replace the quantification with the one the bucket set supports: the ciphertext length identifies which of the thirteen padding buckets the message occupies, which is at most about 3.7 bits of information about the plaintext length, and within a bucket the exact codepoint count is not revealed at all — a message of n codepoints retains roughly $\lceil \log_2 n \rceil - 1$ bits of length uncertainty.

Keep the closing advice that callers needing full length-hiding must layer a fixed-frame transport, which is right.

Client Comment**CVF-12: open**Category: **algorithm**

The size of the prime set is never stated exactly, and the approximation given is wrong, although the sampling rule and the min-entropy figure are both defined in terms of it

The notation section defines the public prime set as the primes in $[10^6, 9.9 \times 10^6]$ and states " $|\mathcal{P}| \approx 586,000$ " (lines 204-206, PDF p. 4). The exact count, which we computed with a sieve, is **579,947**. The stated figure is 6,053 too large.

The quantity is not decorative. The concrete sampling algorithm draws a uniform 32-bit integer, rejects it whenever it is at least $L = \lfloor 2^{32}/|\mathcal{P}| \rfloor \cdot |\mathcal{P}|$, and otherwise returns the value modulo $|\mathcal{P}|$, in order to obtain an exactly uniform index with no modulo bias (lines 226-242, PDF p. 4). Both the rejection threshold and the index range are functions of $|\mathcal{P}|$, so two implementations that disagree about it sample from different distributions and, more to the point, index the canonical ascending enumeration differently — which is the one thing that remark exists to make "identical across implementations". The min-entropy figure is likewise derived from it: with the true count, $\log_2 |\mathcal{P}| = 19.1456$ and $H_\infty(\mathbf{k}) = 191.46$ bits at $K = 10$, with an ordered-tuple key space of $4.30 \times 10^{57} = 2^{191.46}$, against the stated 19.16, 191.6 bits and $4.8 \times 10^{57} \approx 2^{191.6}$ (lines 244-249, PDF p. 4). The post-quantum section's derived figure of $2^{95.8}$ becomes $2^{95.73}$ (lines 962-967, PDF p. 13), and the caveat's " $\approx 586,000$ candidate primes" for the trial-division search becomes 579,947 (lines 1130-1138, PDF p. 15).

The security consequence is nil — 0.15 bits at a level nobody relies on, since no theorem in the paper depends on the arithmetic key's entropy at all. The reason to fix it is reproducibility: the previous round already found these figures numerically wrong, and the correction applied this round propagated a correct formula from an incorrect input, so the numbers are now self-consistent and still not right. An implementer who sieves the interval will get a different count from the one the paper states and will have no way to tell which is normative.

Source anchors (pending artifact upload): lines 204-206, PDF p. 4; lines 226-242, PDF p. 4; lines 244-249, PDF p. 4; lines 962-967, PDF p. 13; lines 1130-1138, PDF p. 15.

Recommendation

State the exact value, $|\mathcal{P}| = 579,947$, as a normative constant rather than an approximation, and say how it is obtained (the count of primes p with $10^6 \leq p \leq 9.9 \times 10^6$), so that an implementation can assert it at startup.

Recompute the four derived figures from it: $\log_2 |\mathcal{P}| = 19.1456$, $H_\infty(\mathbf{k}) \approx 191.46$ bits and key space $\approx 4.30 \times 10^{57} \approx 2^{191.46}$ at $K = 10$, post-Grover $\approx 2^{95.73}$, and the trial-division search space in the caveat.

Consider also publishing the digest of the canonical enumeration, since the sampling remark makes the enumeration itself normative and a hash is the only practical way for an implementation to check it has the right table.

Client Comment

CVF-13: open

Category: **documentation**

The known-answer-test corpus is named for an obsolete version and is described as containing vectors the current construction cannot produce, which undermines it as evidence of cross-language agreement

The known-answer-test section identifies the corpus as `tests/kat/v6_vectors.json` and lists what it covers, including "multiple nonces for the same message" and "same-message determinism" in the same sentence (lines 1070-1082, PDF p. 15). Those two cannot both be true of this construction. The nonce is a deterministic function of the associated data and the message, so for a fixed key there is exactly one nonce per input pair and multiple nonces for one message are unreachable; the previous version's random-nonce construction is where such vectors came from. The file name points the same way: the corpus is named for the version-6 wire format, which this version no longer specifies.

The Rust unit-test list two pages earlier is consistent with the current design — it names round-trip correctness, tamper and associated-data-mismatch rejection, same-message determinism, distinct-message nonce distinctness, and ciphertext-length determinism (lines 1035-1043, PDF p. 14) — so the discrepancy is confined to the corpus description. But the corpus is load-bearing in a way the unit tests are not: the paper offers it as the mechanism by which conforming implementations agree, requiring that they "produce byte-identical ciphertext for every positive vector and raise a matching error for every negative vector", and the conclusion cites it as validation. Evidence offered in those terms has to describe the artifact accurately, and a reader cannot currently tell whether the thirty vectors were regenerated for the current construction or inherited from a version whose nonce was independent of the message. If they were inherited, byte-identical agreement across three languages is being claimed against vectors that the current specification cannot reproduce.

This finding is about the paper's description, which is what the audit scope covers this round; the corpus itself was not submitted and could not be checked.

Source anchors (pending artifact upload): lines 1070-1082, PDF p. 15; lines 1035-1043, PDF p. 14; lines 454-468, PDF p. 7.

Recommendation

Regenerate the corpus against the current construction, rename it away from the version-6 label, and correct the description: drop "multiple nonces for the same message" and, if the intent was to test that distinct associated data yields distinct nonces for one message, say that instead, since it is both true and worth testing.

State in the same paragraph the version of the construction the vectors were generated from, and include the corpus digest, so that a reader can confirm which artifact the byte-identity claim refers to.

Add negative vectors for the decoder checks discussed in CVF-8 while the corpus is being regenerated.

Client Comment

CVF-14: open

Category: **procedural**

The constant-time discussion covers only the tag comparison, while the token loop branches on secret-derived predicates and divides by a secret prime

The section on constant-time considerations is four lines long and addresses one operation: tag comparison, done with `hmac.compare_digest` in Python and `constant_time_eq` from the `subtle` crate in Rust (lines 1050-1054, PDF p. 14). Constant-time tag comparison is necessary and it is good that it is specified. It is not the only secret-dependent control flow in the construction.

The emission loop branches on `is_noise(ct_pos)`, a predicate computed from an HMAC output under the secret key compared against the secret-derived threshold, and it maintains a noise-run counter whose value determines when the loop exits (lines 424-443, PDF p. 7). The number of iterations before each real token is therefore a function of secret material, and an attacker with a local timing or microarchitectural observation channel learns the noise pattern — one bit per token position of the output of a pseudorandom function keyed by the sole secret of the construction. That does not by itself recover the key or the plaintext, but it is a steady leak of keyed pseudorandom-function output, and combined with the trial-division observation the paper itself makes in its caveats, an attacker who learns which token positions are real has removed the only

structural obstacle to reading them. Decryption replicates the same branch structure when inverting the loop (lines 568-580, PDF p. 9), and additionally divides each token by the secret prime k , an operation whose timing is data-dependent on many targets.

The paper is not silent about the gap so much as it defers it: the conclusion lists “a TVLA side-channel analysis of the constant-time Rust implementation” as future work (lines 1178-1180, PDF p. 16). The client materials for this round also include a differential-timing attestation, which suggests the work exists somewhere. But the paper as it stands claims constant-time behaviour for one comparison and says nothing about the two loops, so a reader who sees the section title reasonably concludes the construction has been considered as a whole and it has not.

Source anchors (pending artifact upload): lines 1050-1054, PDF p. 14; lines 424-443, PDF p. 7; lines 568-580, PDF p. 9; lines 1178-1180, PDF p. 16.

Recommendation

Broaden the section from an implementation note into a short threat statement. Say which values are secret-dependent (the noise predicate, the noise-run counter, the addends, the prime selection), which operations touch them, and what an attacker with a local timing channel is assumed not to have.

Then state, for each, whether the reference implementations are constant-time and if not why the leak is judged acceptable — in particular whether the noise pattern is considered secret, since the whole traffic-analysis rationale for the layer assumes it is. If the branch on the noise predicate is intended to be secret-independent, that needs an argument; if it is not constant-time, say so plainly and move the deferred differential-timing analysis into scope rather than into future work.

Referencing the existing timing attestation from the paper would also let a reader find the evidence that does exist.

Client Comment

CVF-15: open

Category: **documentation**

The abstract’s central differentiator — that unlike the established schemes NAPQES is immune to Shor-style quantum attacks — is contradicted by the paper’s own comparison table

The abstract states that, unlike AES-GCM and ChaCha20-Poly1305, NAPQES employs no finite-field arithmetic, group operations or S-box permutations, and is “therefore not subject to Shor’s algorithm or hidden-subgroup quantum attacks” (lines 54-59, PDF p. 1).

The introduction develops the same contrast, saying the established schemes' security "ultimately depends on algebraic structures ... whose resilience against quantum adversaries running variants of Shor's algorithm is an active research question" (lines 94-99, PDF p. 3). The rhetorical structure of both passages is that the comparison schemes are exposed to a quantum threat that NAPQES avoids.

The comparison table settles the question in the other direction. Its row "Shor-applicable structure" reads "No" for NAPQES, "No" for AES-GCM, "No" for ChaCha20-Poly1305 and "No" for Ascon (line 989, PDF p. 14). That is correct — Shor's algorithm attacks the hidden-subgroup problems underlying factoring and discrete logarithms, and neither the polynomial arithmetic in GHASH and Poly1305 nor an add-rotate-xor round function presents such a structure — and it means the differentiator claimed in the abstract does not exist. The paper's own analysis section is careful and correct on the point that matters: the only quantum speedup in play against any symmetric scheme here is Grover's, which halves the effective key-search exponent for all of them alike (lines 947-950, PDF p. 13).

The paper is therefore right in its analysis and wrong in its framing, and the framing is what a reader takes away from an abstract. Since the construction has real properties to advertise — a single approved primitive, no polynomial authenticator and hence no forbidden-attack exposure, and graceful degradation under nonce misuse, all of which the table also records — the unsupportable quantum claim is not even load-bearing for the paper's case.

Source anchors (pending artifact upload): lines 54-59, PDF p. 1; lines 94-107, PDF p. 3; line 989, PDF p. 14.

Recommendation

Rewrite the two passages so they claim what the table supports. The defensible statements are that the construction has no polynomial or field-arithmetic authenticator and so no forbidden-attack exposure under nonce misuse; that it uses a single primitive, reducing the assumption surface to one well-studied function; and that its post-quantum posture, like that of every symmetric scheme considered, is governed by Grover's algorithm and nothing stronger.

Drop the suggestion that the comparison schemes are Shor-exposed, or, if the intended claim was about the long-run risk of cryptanalysis against algebraic structure rather than about Shor's algorithm specifically, say that instead and separate it clearly from the quantum argument.

Client Comment**CVF-16:** openCategory: **documentation****NIST's transition guidance is misstated: 112-bit security is being withdrawn rather than recommended as the post-2030 minimum**

The post-quantum section concludes its treatment of Grover's algorithm by observing that at 2^{128} sequential oracle queries the scheme "remains far beyond feasibility and above the 112-bit minimum NIST recommends for post-2030 systems" (lines 943-945, PDF p. 13). That inverts the guidance. NIST's transition schedule treats 112-bit security strength as acceptable only *through* 2030 and deprecates it thereafter, with a view to disallowing it entirely; the minimum recommended for systems intended to operate after 2030 is 128 bits, not 112. Citing 112 bits as the post-2030 floor turns a level being retired into a level being endorsed.

The scheme comfortably exceeds either threshold, so nothing in the security argument depends on this and the severity is low. It is worth correcting because the sentence is doing rhetorical work — it is the paper's one appeal to an external standard for the adequacy of its post-quantum margin — and because a claim about a standards body's recommendation is the kind of statement a reader will take on trust rather than check.

The surrounding treatment of Grover's algorithm is otherwise good and better than most: the distinction between sequential query depth and parallelisable work factor is correct, and the appeal to the optimality result of Bennett, Bernstein, Brassard and Vazirani and to Zalka's parallelisation bound is apt and correctly characterised — distributing Grover's search across S machines does yield only a $\sqrt{S}$ speedup. This finding is confined to the standards claim in the final sentence.

Source anchors (pending artifact upload): lines 931-945, PDF p. 13.

Recommendation

Replace the clause with an accurate statement of the guidance: that NIST's transition plan deprecates 112-bit security after 2030 and sets 128 bits as the minimum for systems expected to remain in service beyond it, so a 2^{128} sequential-depth bound meets the post-2030 minimum rather than exceeding a 112-bit floor.

Cite the specific publication relied on, since the paper's bibliography currently contains no NIST transition document and the reader has no way to check the claim.

Client Comment**CVF-17:** openCategory: **documentation**

Several citations do not support the statements attached to them, two relied-upon standards are absent from the bibliography, and one competitor's standardisation status is out of date

Four defects, all in the citation apparatus.

A misattributed reference. The related-work discussion of AES-GCM cites "the Forbidden Attack and nonce-reuse attacks" to two keys, the second of which is named Joux-nonce (lines 154-161, PDF p. 3). That key resolves to Iwata, Ohashi and Minematsu, "Breaking and Repairing GCM Security Proofs", CRYPTO 2012 (lines 1226-1229, PDF p. 17). That paper is about an error in GCM's security proof and its repair, not about nonce-reuse attacks, and it is not by Joux — so both the key name and the claim it is attached to point at the wrong work. The neighbouring citation to Joux's 2006 comment on authentication failures in GCM is correct and is the right reference for the Forbidden Attack.

Two missing references. The synthetic nonce is introduced as "a synthetic IV in the manner of RFC 5297 SIV / AES-GCM-SIV" (lines 463-468, PDF p. 7), the related-work section says AES-GCM-SIV "follows the same synthetic-IV philosophy NAPQES adopts" (lines 180-184, PDF p. 3), and the comparison section returns to the analogy (lines 1011-1019, PDF p. 13). Neither RFC 5297 nor AES-GCM-SIV appears anywhere in the fifteen-entry bibliography. These are the two works the design most directly claims kinship with, and the claim is the basis of the misuse-resistance argument, so they are the least appropriate references to omit.

An out-of-date status claim. The related-work paragraph on Ascon says it "is not yet in a FIPS-approved family" (lines 169-173, PDF p. 3) and the comparison table records its approval status as "No (pending)" (line 988, PDF p. 14). NIST published SP 800-232 standardising the Ascon-based lightweight cryptography family in 2025, before this paper's stated date. By the criterion the table itself applies — AES-GCM is marked approved on the strength of SP 800-38D — Ascon now qualifies on the same footing.

A superseded reference. ChaCha20-Poly1305 is cited to RFC 7539, which was obsoleted by RFC 8439 in 2018.

Source anchors (pending artifact upload): lines 154-161, PDF p. 3; lines 169-173, PDF p. 3; lines 463-468, PDF p. 7; line 988, PDF p. 14; lines 1191-1194, PDF p. 16; lines 1226-1229, PDF p. 17.

Recommendation

Point the nonce-reuse claim at a reference that supports it, or drop the claim and keep the Forbidden Attack citation, which is sound; and rename the key so it no longer attributes the proof-repair paper to Joux.

Add bibliography entries for RFC 5297 and for AES-GCM-SIV (RFC 8452) and cite them at each of the three places the analogy is drawn.

Update the Ascon status in both the related-work paragraph and the comparison table, noting SP 800-232, and check the rest of that column against the same criterion. Replace RFC 7539 with RFC 8439.

Client Comment**CVF-18:** openCategory: **documentation**

The comparison table's key size of at least 82 bytes is not a lower bound, since the number of primes is a free parameter and the paper's own test corpus exercises a single-prime key

The comparison table gives NAPQES a key size of " ≥ 82 bytes ($k+sk$)" against 32 bytes for AES-GCM and ChaCha20-Poly1305 and 16 for Ascon (line 993, PDF p. 14). The figure is the $K = 10$ case: ten primes serialised as five bytes each, plus the 256-bit secret, is $50 + 32 = 82$. It is not a minimum, because K is a parameter with no stated range. Key generation introduces K with a default of 10 and no constraint (lines 214-224, PDF p. 4); the remark on key roles states that " K therefore carries no security-theorem-driven minimum in this construction" and that 10 is retained only as a default (lines 251-270, PDF p. 5); and the known-answer-test section says the corpus covers "1-element and 10-element keys" (lines 1074-1078, PDF p. 15). A one-prime key is therefore both permitted and tested, and its serialised size is 37 bytes, well below the stated floor.

This is minor in itself, but it sits on top of a genuine gap: the paper never states the permitted range of K anywhere. A conforming implementation cannot tell whether $K = 1$ is legal, whether there is an upper limit, or what a decryptor should do when handed a key of unexpected length — and K is not transmitted in the ciphertext, so both parties must agree on it out of band. The addend derivations reduce modulo $k - 1$, which requires $k \geq 3$ and is satisfied by every prime in the set, so no lower bound arises from the arithmetic; the constraint, if any, has to be stated as policy.

Source anchors (pending artifact upload): line 993, PDF p. 14; lines 214-224, PDF p. 4; lines 251-270, PDF p. 5; lines 1074-1078, PDF p. 15.

Recommendation

State the permitted range of K in the key-generation definition, with the default, and say whether values other than the default are conformant.

Then either correct the table entry to the exact key size at the default $K = 10$, namely 82 bytes, noting that it scales as $5K + 32$, or give the true range across permitted K .

Since K is not carried in the ciphertext, add a sentence saying it is a negotiated or configured parameter that both parties must agree on, and state what a decryptor does if the configured K does not match the one used to encrypt.

Client Comment**CVF-19:** openCategory: **readability**

The abstract and the contributions list misstate the construction and promise material the paper does not contain

Three mismatches between the front matter and the body.

The nonce formula is given without its truncation. Both the abstract (lines 63-66, PDF p. 1) and the first contribution (lines 111-118, PDF p. 3) write the synthetic nonce as $N = \text{HMAC}(\text{sk}, 0x0A\|\text{be8}(|A|)\|A\|M)$, with no truncation, while the normative definition takes the first sixteen bytes and the wire format carries a sixteen-byte nonce. As written, the front-matter formula describes a 32-byte nonce, which is not the construction. This is the same class of defect as the previous round's finding that the abbreviated formula omitted the length prefix: that omission was corrected, and the prefix is now present in both places, but the truncation was not carried across at the same time.

The cross-references in the third contribution are scrambled. It reads "We prove IND-CPA-det (Definition ...), INT-CTXT (Theorem ...), and IND-CCA-det (Theorem ...) security (Theorem ...)" (lines 123-126, PDF p. 3): the first of three parallel results points at a definition rather than the theorem proving it, and a fourth reference to the confidentiality theorem dangles at the end of the sentence, after the list has closed.

The abstract promises measurements the paper does not report. It says the authors "provide performance measurements and implementation notes across Python, Rust, and C reference implementations" (lines 79-83, PDF p. 1). The implementation section contains implementation notes and no measurements — no timings, no throughput, no platform, nothing quantitative (lines 1022-1054, PDF p. 14) — and the conclusion lists "a performance evaluation on embedded ARM Cortex-M4 and RISC-V targets" as future work

(lines 1178-1180, PDF p. 16). The abstract and the conclusion therefore contradict each other about whether performance has been measured.

Source anchors (pending artifact upload): lines 63-66, PDF p. 1; lines 109-147, PDF p. 3; lines 79-83, PDF p. 1; lines 1022-1054, PDF p. 14; lines 1178-1180, PDF p. 16.

Recommendation

Add the sixteen-byte truncation to both front-matter statements of the nonce formula, so the informal version matches the definition.

Repair the third contribution's references so each claimed result points at the theorem that proves it, and remove the trailing reference.

Then reconcile the performance claim: either add the measurements, which would strengthen the paper given that expansion is its principal cost, or delete the promise from the abstract and leave the evaluation in future work, where the conclusion already puts it.

Client Comment

CVF-20: open

Category: **procedural**

Statistical-test, known-answer and fuzz results are presented as validation of the security claims, which they cannot support

The conclusion states that the scheme "has been validated against a 30-vector Known-Answer Test corpus, the full NIST SP 800-22 statistical suite, and a fuzz harness covering the complete decoder pipeline" (lines 1169-1172, PDF p. 16), and the abstract leads with the statistical analysis alongside the security theorems. None of the three bears on the properties the paper proves. Known-answer vectors establish cross-implementation agreement, which is valuable and is what the paper should claim for them. Fuzzing establishes that the decoder does not crash on malformed input, which is also valuable and also not a confidentiality or integrity property. Statistical testing of ciphertext establishes neither.

The statistical section reports that the full NIST SP 800-22 suite was applied to 10^7 bits of ciphertext and that all fifteen eligible tests passed at the 0.01 level (lines 1057-1067, PDF p. 14). This result was a foregone conclusion and carries essentially no information: the token blob is masked with a keystream derived from HMAC-SHA256, so the tested bits are the output of a pseudorandom function exclusive-ored with structured data, and any keystream that failed a general-purpose randomness suite would already be a break of the underlying primitive rather than a finding about this scheme. Passing tells us nothing

we did not assume; and because the suite would equally pass on the keystream alone, it cannot distinguish a construction that hides the plaintext from one that does not. NIST itself positions SP 800-22 as a tool for evaluating random and pseudorandom generators, not as evidence for a mode's security.

The concern is not that the testing was done — all three activities are worth doing and worth reporting — but that the conclusion's word "validated", applied to a scheme whose security rests on three theorems, invites a reader to count them as corroboration of those theorems. A reader who is not going to check the proofs is exactly the reader most likely to be reassured by a list of suites that passed.

Source anchors (pending artifact upload): lines 1057-1067, PDF p. 14; lines 1169-1172, PDF p. 16; lines 79-83, PDF p. 1; lines 1085-1096, PDF p. 15.

Recommendation

Say what each activity establishes. The known-answer corpus establishes byte-level agreement between the Python, Rust and C implementations and should be cited for interoperability. The fuzz harnesses establish decoder robustness against malformed input, with no panics over the reported corpus. The statistical suite establishes that the masked output shows no gross statistical structure, which is a sanity check on the implementation of the keystream rather than evidence for any security claim, and the section should say so explicitly.

Reserve "validated" for the properties actually established, and keep the security claims resting on the theorems, whose bounds are the paper's real contribution.

Client Comment

CVF-23: open

Category: **documentation**

The modulo-bias remark justifies the absence of rejection sampling with a domination argument that runs backwards, and exempts the noise threshold from a bias it actually carries

The numeric figures in "Remark 3.6, Modulo bias" are correct (lines 392-408, PDF p. 7): reducing a 32-bit pseudo-uniform value modulo m skews each residue's probability by a relative factor of at most $m/2^{32}$, which is below 2^{-25} for the padding codepoint and noise character and below 2^{-8} for the addend moduli. The security inference drawn from them is unsound in two independent ways.

First, the addend distribution sits at total variation distance $\frac{s(m-s)}{m \cdot 2^{32}} \leq \frac{m}{2^{34}} \approx 2^{-11}$ from uniform per sample — a fixed constant — while "Assumption 4.2, Uniform-key HMAC-SHA256 PRF" asserts that the PRF advantage is *negligible*. A constant of order 2^{-11} cannot be "statistically dominated by" a negligible quantity, so the stated domination is exactly inverted.

Second, the exemption claimed for the noise threshold is false. $\theta(N)$ is not computed from fixed public constants alone but from the pseudorandom 64-bit draw τ_i and the fixed-point scaling $\lfloor \tau (\theta_{\max} - \theta_{\min}) / 2^{64} \rfloor$ over a range of size $\approx 0.24 \cdot 2^{64}$ has the same floor-versus-ceiling non-uniformity as any modulo reduction — here with per-value relative deviation of about +20% and total variation distance $\approx 0.033 \approx 2^{-4.9}$. That is the largest bias anywhere in the construction, sitting in the one value the remark declares bias-free.

The remark's bottom-line conclusions happen to be true, but for a reason it never gives: the token blob is masked by the domain $0x07$ keystream before serialisation, and the "Hiding" lemma applies the one-time-pad identity for a *fixed* blob, so the masked ciphertext is uniform however biased the addends, characters, padding codepoints or threshold may be, while the integrity theorems depend only on the tag. As written the remark would license a reader or reimplementer to treat a 2^{-11} -to- 2^{-5} statistical bias as generically absorbed by a PRF assumption, an argument that fails immediately in any variant where these reduced values are exposed unmasked.

Exploitability. We worked the question of whether a powerful adversary can turn these biases into an attack, and the answer is a definite no rather than a "probably fine". Every biased quantity is confined to the token blob, which is a one-time pad under the keystream, so the exclusive-or of a biased blob with an independent uniform mask is uniform for an arbitrary blob; the distinguishing, arithmetic-key-recovery and plaintext-recovery paths therefore contribute advantage exactly zero. The single many-sample attack that does work — a two-time-pad break once the 128-bit synthetic nonce repeats at $q \approx 2^{64}$ — is the nonce-truncation finding recorded in CVF-21, priced by the $q^2/2^{128}$ term of the confidentiality theorem, and the addend bias cancels in that exclusive-or rather than assisting it.

Severity. This was rated low after the severity was specifically questioned during review. The escalation test applied throughout this engagement raises a finding to at least moderate when there is a gap in a proof, and here there is none: "Lemma 4.9, Hiding" is stated and proved for a *fixed* blob, so it is bias-agnostic, and no theorem anywhere invokes uniformity of the addends, characters, padding codepoints or the threshold. No numeric bound in the security section changes whether or not the bias is corrected, and a clear local repair exists. The finding is therefore a documentation-soundness defect and stays at low — recorded here so that the disagreement, and the reason it resolved as it did, is on the record rather than lost.

Source anchors (pending artifact upload): lines 392-408, PDF p. 7; lines 340-355, PDF p. 6.

Recommendation

Replace the domination sentence with the correct justification — the biased values never enter the adversary's view because the masked blob is a one-time pad under a fresh nonce, so no bound in the security section contains or needs a bias term —

and either delete the claim that the bias is dominated by the PRF advantage or invert it into an honest caveat that the bias is non-negligible in isolation and is rendered unobservable only by the masking.

Correct the noise-threshold paragraph to acknowledge that fixed-point scaling of the raw draw carries the same, larger non-uniformity, and that it is likewise harmless only because the threshold is never exposed and no theorem requires it to be uniform.

It is worth stating the boundary of safety explicitly, since it is narrow: harmlessness rests entirely on the masking, so any variant that exposes a reduced value before the pad, drops the pad, or reuses a nonce makes the addend and character bias immediately relevant.

Client Comment

CVF-24: open

Category: **procedural**

The consecutive-noise cap keeps ciphertext length independent of the noise realisation, but encryption and decryption time still varies with the secret-derived threshold and with the exact codepoint count, and the constant-time discussion never mentions it

The cap and ceiling-padding construction of the token-construction section is sound as a length equaliser: the main loop emits at most 20 tokens per real codepoint, so the filler loop always tops the count up to exactly $20R$ tokens, and "Lemma 3.8, Ciphertext length is a deterministic function of the padding bucket" holds. The variability it removes from the wire, however, reappears in execution time.

Genuine noise tokens and filler tokens cost the same two derivations each and their totals cancel, but the number of noise-oracle evaluations is $\sum_i \min(r_i + 1, 19)$, so the token loop performs between $40R$ and $58R$ derivations, with expectation about $43R$ at threshold $p = 0.75$ and about $56R$ at $p = 0.99$ — a measurable swing of roughly 30% driven entirely by a per-message threshold that is a keyed function of the nonce. A second, sharper dependency sits in the padding step, which performs exactly $B - n$ derivations and therefore varies with the exact codepoint count rather than merely with its bucket, which is more than the padding-bucket caveat concedes. The "Constant-Time Considerations" subsection addresses only the tag comparison, so the specification silently presents a data-dependent running time.

The impact is limited, and we say so plainly: the modulated quantity in the token loop is a pseudorandom function of a nonce that is transmitted in the clear anyway, and decryption's variable phase runs only after successful tag verification, so no forgery timing

oracle arises. A co-resident timing adversary gains at most a per-message fingerprint and, through the padding loop, the exact plaintext length rather than only its bucket.

Separately, the decryption algorithm says only to “invert the token-emission loop”. A decoder that consults the noise-position oracle without also replicating the run cap would misclassify a cap-forced real token, which at $p = 0.99$ happens with probability about 0.83 per real token, so the cap-replication requirement deserves to be stated rather than implied.

Source anchors (pending artifact upload): lines 416-449, PDF pp. 6-7; lines 1050-1054, PDF p. 14; lines 577-579, PDF p. 9.

Recommendation

Add a caveat stating that running time, unlike ciphertext length, depends on the per-message noise threshold and its realisation and on the exact codepoint count through the padding loop, together with the argument for why this is acceptable under the stated threat model.

Extend the constant-time subsection to cover the token and padding loops rather than only the tag comparison, or say plainly that they are not constant-time.

State explicitly in the decryption algorithm that the inversion must reproduce the consecutive-run cap, not merely the raw noise-position oracle.

Client Comment

CVF-25: open

Category: **readability**

None of the three security theorems names the scheme it is about, and their advantage functionals carry no scheme parameter, unlike the PRF advantage they are stated against

The confidentiality theorem reads “For any PPT adversary $\mathcal{A}$ respecting the restrictions and making at most q encryption queries, there exists a PRF adversary $\mathcal{B}_1$ such that $\text{Adv}^{\text{IND-CPA-det}}(\mathcal{A}) \leq \dots$ ” and never states which scheme is being attacked. The same holds for the integrity theorem and the chosen-ciphertext theorem, and for the advantage defined in “Definition 4.4, IND-CPA-det security”.

The contrast with the paper’s own notation is sharp: the PRF advantage is consistently written $\text{Adv}_F^{\text{PRF}}(\mathcal{B})$ or $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}(\mathcal{B})$, correctly carrying the function it is taken against, so the omission on the scheme side reads as an oversight rather than a convention. The scheme enters only indirectly, through the definition’s reference to the

encryption oracle, which means that a reader lifting a theorem statement out of context — as happens when a bound is quoted in a survey, a comparison table, or a downstream security argument — obtains a statement with no subject.

The paper also mixes the two spellings of the same object within one proof, writing $\text{Adv}_{\text{HMAC-SHA256}}^{\text{PRF}}$ in the theorem and $\text{Adv}_F^{\text{PRF}}$ in the final display. This is a presentation defect with no effect on any bound, but it is the kind that a careful reader stops on, and it is cheap to fix.

Source anchors (pending artifact upload): lines 683-708, lines 732-745, lines 829-843, lines 883-898; PDF pp. 10-12.

Recommendation

Parameterise every advantage by the scheme and by the query budget — for example write the confidentiality advantage as $\text{Adv}_{\Pi}^{\text{IND-CPA-det}}(\mathcal{A})$ for the named triple, and likewise for the integrity and chosen-ciphertext advantages — and open each theorem statement with the scheme it concerns.

Fix the notation on a single spelling for the HMAC PRF advantage and use it consistently in statements and proofs.

Client Comment

6. Conclusion

The third-round review of the NAPQES ePrint identified twenty-five issues: one of high severity, eight of moderate severity, and sixteen of low severity. Of the nineteen findings carried into this round — the fourteen raised against version 2 and the five raised in the first round that were still open or acknowledged — eighteen are fixed and one is partially fixed; none is unresolved and none has regressed.

That result should be stated plainly: version 3 repairs every soundness defect the previous rounds identified in the security analysis. The confidentiality theorem is now proved against a restricted notion that a deterministic scheme can actually satisfy; the reduction genuinely simulates the encryption oracle, because the derivation secret is sampled independently of the arithmetic key, which closes the first round's simulation gap without recourse to a non-standard key distribution; the PRF assumption and its advantage are stated formally, and the truncated derivations are accounted for; the ciphertext length is proved to depend only on the padding bucket, which kills the averaging oracle the second round demonstrated; and the legacy construction, the streaming format and the entire embedded audit-response narration have been removed, leaving a document that can be read on its own merits. Each of the paper's proofs and its one proof sketch was reduced to its inference skeleton and re-derived independently, rather than accepted on the strength of the paper's own account of its repairs, and the load-bearing arguments hold. The one exception is not a relapse but an unfinished migration, and it is the subject of the single high-severity finding.

That finding began as a specification defect and proved to be more. The widening of the associated-data length prefix from four to eight bytes — itself a first-round repair — was applied to four of its six sites, and the two that were missed are the decryption algorithm's tag input and the tag input in the ciphertext-integrity proof. As written, the specified decryptor recomputes a different tag from the one the specified encryptor produces, so no honest ciphertext verifies and the stated correctness property is false. The same width disagreement is also an authentication break. Because the decryptor consumes a four-byte length field where the encryptor emitted an eight-byte one, the four leading zero bytes of the longer field can be re-attributed out of the length and into the body, and an adversary who makes a single encryption query on associated data of a suitable length can re-split the answer into a fresh ciphertext-and-associated-data pair that the specified decryptor accepts with probability approximately one — against an asserted bound of $\text{Adv}^{\text{PRF}}(\mathcal{B}_2) + q_v/2^{256}$. The integrity theorem is therefore false for the scheme as literally written, not merely unsupported by its proof. One edit in each of two places closes it, and the repair must widen rather than narrow, since the eight-byte form is what the domain-separation lemma proves injective and what the synthetic-nonce derivation

consumes; we verified the counterfactual, and after those two edits the theorem is true as stated, bound included. The reference implementations should then be checked against the corrected text, since the three of them cannot all agree with a document containing both widths, and any implementation that follows the specified decryptor literally inherits the forgery.

The moderate findings concern what the paper claims rather than what it proves. The noise-injection and prime-token layer is required by no theorem and carries no stated property — the paper's own caveats concede that compromise of the derivation secret alone is a full break, recoverable by trial division in microseconds — yet that layer is the sole cause of the scheme's ciphertext expansion, and the one justification offered for the default number of primes cites a lemma that proves the opposite of what is wanted. The expansion is itself misstated: quoted as a fixed twenty-fold in four places, it is one hundred and sixty-fold per plaintext codepoint on the wire, rising to roughly three hundred and twenty-fold just past a padding-bucket boundary, as the paper's own 2,928-byte minimum ciphertext already implies. The confidentiality notion actually proved is a single-challenge, no-repeat game, weaker than the deterministic-authenticated-encryption notion the paper invokes by name and weaker than the notion AES-GCM-SIV is analysed under. Chosen-ciphertext security is claimed in the abstract, the contributions and the conclusion but is never defined, and rests on a proof sketch that attributes to Bellare and Namprempre a composition result their paper does not state. Ciphertext integrity is likewise bounded without ever being defined: no integrity experiment appears anywhere in the document, so the inequality the theorem states has no game to be checked against, and the proof's first case cites the domain-separation lemma outside the scope that lemma is stated for. The comparison section denies the birthday-bound nonce collision that the paper's own confidentiality theorem prices at $q^2/2^{128}$, calling accidental reuse a negligible hash-collision event rather than what it is, with the consequence that a per-key data limit of roughly 2^{64} messages goes undisclosed and a reader of that section would conclude no such limit applies. The message space is defined in Unicode codepoints while the nonce is computed over UTF-8 bytes and the reference interface takes bytes, with no mapping specified between the three. And the comparison table asserts that NAPQES itself is FIPS-approved, which conflates the approval status of a primitive with that of a mode built from it.

The low-severity findings are dominated by unvalidated decoder inputs; by figures that are missing or quantified backwards, among them the size of the prime set, the padding-bucket leak and the stated key size; by a known-answer-test description that is stale and incompatible with the synthetic-nonce construction, which matters because that corpus is the paper's evidence of cross-language agreement; by a constant-time discussion that covers only the tag comparison while both token loops branch on secret-derived predicates and divide by a secret prime; and by several claims about quantum resistance, standards guidance and cited literature that the paper's own comparison table or the cited works do not support. Three further minor findings concern the paper's account of its own randomness and timing behaviour. The modulo-bias remark justifies the absence of rejection sampling with a domination argument that runs in the wrong direction, and exempts the noise threshold from what is in fact the largest non-uniformity in the construction; we worked the exploitability question directly and found the answer to be a definite no, because every biased value sits behind the keystream mask and no theorem requires any of them to be uniform, but the remark's stated reasons are not the reasons

that hold. The consecutive-noise cap equalises ciphertext length yet leaves running time dependent on a secret-derived threshold and, through the padding loop, on the exact codepoint count rather than merely its bucket, which the constant-time discussion does not mention. And none of the three security theorems names the scheme it is about, in a document that is otherwise careful to parameterise the PRF advantage by the function it is taken against.

Our overall assessment is that the security analysis of version 3 is, for the first time in this engagement, structurally sound: the reductions are genuine, the hybrid arguments close, and we found no proof in the document whose repair requires rethinking the design. What the document does not yet have is a specification that its proofs describe. The tag-input inconsistency makes the integrity theorem false as written while leaving it true after a two-line correction; the integrity and chosen-ciphertext notions are asserted without being defined; and the expansion figure, the per-key message limit and several supporting claims are stated in terms that overstate what has been established. Once those are corrected, the document would substantiate what it sets out to prove. We continue to recommend that the authors state plainly what the noise and prime-token layer buys, since a design whose dominant cost is attached to no proven property is unlikely to survive external review.

Client Comment



ABDK

Consulting

About Us

Established in 2016, is a leading service provider in the space of blockchain development and audit. It has contributed to numerous blockchain projects, and coauthored some widely known blockchain primitives like Poseidon hash function.

The ABDK Audit Team, led by Mikhail Vladimirov and Dmitry Khovratovich, has conducted over 300 audits of blockchain projects in Solidity, Rust, Circom, C++, Javascript, and other languages.

Contacts

Email

info@abdkconsulting.com

Website

abdk.consulting

X (former Twitter)

[@ABDKConsulting](https://twitter.com/ABDKConsulting)

LinkedIn

company/abdk-consulting